

would be — the CSS behind the two pages

Every layout tool used in `Wouldbe.jsx`, `StartAnOffice.jsx`, `StartAWouldBe.jsx` and their components — what it does, the exact commands, where it lives in the real code, and the trap that bit us. Pair each section with its `css/NN-*.html` practice page.

1. The mental model (cascade + box model)
2. Units: px · % · vw · em · rem · fr · clamp
3. Flexbox — the split & the columns
4. Grid — proportional tracks
5. Positioning + centering + z-index
6. Overflow — the clipping bug
7. Custom properties (the JS→CSS boundary)
8. Gradients — the progress rail
9. Transitions & :hover
10. Selectors, specificity & scoping
11. Typography & text wrapping
12. Pseudo-elements — styling the slider
13. Borders, radius, shadows, pills
14. filter vs backdrop-filter
15. Media queries
16. Map of the real files

1 • The mental model

The cascade & specificity

When two rules touch the same element, the winner is decided by (1) importance, (2) **specificity** — an inline style beats an id beats a class beats a tag — then (3) source order (last wins on a tie). Two of your bugs were cascade bugs: `.blurred` defined in three files all hitting one element, and an inline `align-items: baseline` quietly overriding a class's `center`.

The box model

Every element is a box: `content → padding → border → margin`. Widths get confusing because by default `width` means the *content* width, so padding+border are added *on top*. Fix it globally so `width` means the whole box:

```
/* used at the top of StartAWouldBe.css */
* { box-sizing: border-box; }
```

In the code: `box-sizing: border-box` on the scoped root so padding never blows a panel past its column width.

2 · Units

Unit	Means	Used for
px	absolute pixels	borders, small fixed sizes, pin sizes
%	percent of the <i>parent</i>	rail insets (<code>left: 5.5%</code>), widths
vw / vh	1% of viewport width / height	the fluid part of <code>clamp()</code> ; full-screen heights
em	x the element's own font-size	subtitle <code>0.55em</code> , logo <code>1.1em</code>
rem	x the <i>root</i> font-size	section headers (<code>1.1rem</code>)
fr	a share of grid free space	the 3 columns <code>305fr 280fr 377fr</code>

```
/* fluid, but fenced: min 28px, grows with the window, never past 46px */
font-size: clamp(28px, 4vw, 46px);
```

In the code: the headline uses `clamp()` ; the gray subtitle is `0.55em` of it, so shrinking the headline shrinks the subtitle with it. **PRACTICE 06 + 07**

3 · Flexbox — one-dimensional layout

Turn a container into a flex row/column, then each child gets a `flex` shorthand: `flex: <grow> <shrink> <basis>` .

```
/* the WouldBe two-column split */
.mainContainer { display: flex; gap: 32px; }
.regulationContainer { flex: 0 0 360px; } /* fixed: no grow/shrink, 360px */
.wouldBeRowsMainContainer { flex: 1 1 0; min-width: 0; } /* fill the rest */
```

Alignment: `align-items` is the cross axis (`stretch` makes children equal height), `justify-content` is the main axis (`space-between` pushes first to one end, last to the other). `margin-left: auto` on one child eats the free space and shoves it to the far edge (how the nav's auth buttons sit right).

The trap: a flex item's default is `min-width: auto` — it *won't* shrink below its content. A long word/URL then forces the column wider. Add `min-width: 0` to let it shrink (then `overflow-wrap` breaks the text). **PRACTICE 01**

4 · Grid — two-dimensional / proportional tracks

Grid defines columns up front. `fr` units hold a *ratio* at any width — no vw math, no media query to keep proportions.

```
.grid {
  display: grid;
  grid-template-columns: 305fr 280fr 377fr; /* keeps the ratio */
  gap: 22px;
  align-items: stretch; /* all columns = tallest one */
}
```

In the code: the StartAWouldBe 3-up. When the column count is dynamic (the timeline), the template is set *inline* from React: `repeat({n}, 1fr)` — a static `.css` can't know `n`. **PRACTICE 02**

5 · Positioning & centering

value	behavior
<code>static</code>	normal flow (default)
<code>relative</code>	stays in flow, but becomes an anchor for absolute children; can nudge with top/left
<code>absolute</code>	out of flow, placed against the nearest positioned ancestor
<code>sticky</code>	flows until it hits <code>top:0</code> , then pins while the container scrolls
<code>fixed</code>	pinned to the viewport

```
/* center a marker ON a point: edge to the point, then pull back half its size */
.marker { position: absolute; left: 40%; transform: translateX(-50%); }
```

z-index only reorders *overlapping, positioned* boxes (who paints on top). It is not about clipping — see §6.

In the code: the timeline card is `relative`; the rail, pins, the rose YOU flag and the corner goal chip are all `absolute` inside it. The nav is `sticky`. **PRACTICE 03 + 07**

6 · Overflow — and the clipping bug you hit

`overflow` controls what happens to content bigger than its box: `visible` (spills, default), `hidden` (clipped), `auto/scroll` (scrollbars). Two axes: `overflow-x`, `overflow-y`.

The bug: the regulations text was cut off. Cause: an inner block had `min-width: 70vw`, far wider than its ~32% column, and the column had `overflow-y: auto`. **Setting one axis to auto forces the other from visible to auto too** — so the too-wide content was clipped on the x-axis. `z-index` can't fix clipping. The fix was `min-width: 0 + width: 100%` + wrapping the URL.

7 · Custom properties — the JS → CSS boundary

A variable is `--name: value`, read with `var(--name, fallback)`. They cascade and inherit, so React can set one inline and your stylesheet reads it. This is the clean seam between "compute in JS" and "paint in CSS".

```
/* React sets the number: */ <div className="rail" style={{ '--progress': `${progress}%` }} />
/* CSS consumes it (see §8) */
```

Design tokens work the same way — `--gold`, `--ink`, `--line` declared once on the scoped root, reused everywhere, so a palette change is one edit.

In the code: `--progress` drives the rail; the color tokens live on `.StartAWouldBeMainContainer` and `.wbNav`.

PRACTICE 04

8 · Gradients

`linear-gradient(angle, stops...)` paints along a line. The trick for a hard progress bar: put **two stops at the same position** so there's no blend — a crisp edge.

```
background: linear-gradient(90deg,
  #c9a437 0%, #c9a437 var(--progress), /* gold up to progress */
  #e7e7ea var(--progress), #e7e7ea 100%); /* gray after — hard edge */
```

`radial-gradient` makes the soft warm glow behind the hero (`radial-gradient(1200px 600px at 82% -8%, ...)`).

In the code: the timeline rail (both the StartAWouldBe and the hover card) and the hero background. PRACTICE 04

9 · Transitions & :hover

`transition: <prop> <duration> <easing>` animates a change instead of snapping. Put it on the **base** rule so it eases in *and* out. `:hover` is a pseudo-class — only active while the pointer is over the element.

```
.wouldbeCard { height: 130px; transition: height 0.2s ease; }
.hoveredWouldBeCard { height: 165px; transition: height 0.2s ease; overflow: hidden; }
```

In the code: the card grows on hover. React swaps the class, but it's the same DOM node, so putting `transition` on both classes animates either way. PRACTICE 05

10 · Selectors, specificity & scoping

Component CSS leaks globally unless you scope it. A bare `input[type=range] { ... }` or `button { ... }` restyles every such element in the app. Prefix every rule with a component root:

```
.StartAWouldBeMainContainer input[type=range] { ... } /* scoped: can't leak */
.wbNav-qualify { ... } /* component-prefixed class */
```

The trap: `.blurred` was defined in three stylesheets and all loaded on the page, so the winner was just "last file in". Consolidating to one definition (and scoping the rest) is why the consolidation pass mattered. **PRACTICE 06**

11 · Typography & text wrapping

Key properties: `font-family`, `font-size`, `font-weight` (400 normal / 700–800 bold), `font-style` (`italic` — not `bold`); a real bug we fixed was `font-style: bold`), `line-height`, `letter-spacing`, `text-transform`.

```
/* let a long unbreakable URL wrap instead of forcing the column wider */
a { word-break: break-word; overflow-wrap: anywhere; }
```

In the code: the eligibility source URL in Regulations; the clamp headline. **PRACTICE 01 + 07**

12 · Pseudo-elements — styling a range input

Form controls are OS-drawn until you opt out with `appearance: none`. Then the `<input>` is the track and the knob is a per-engine pseudo-element.

```
input[type=range] { -webkit-appearance: none; appearance: none; height: 6px; background: #3a3a40; }
input[type=range]::-webkit-slider-thumb { -webkit-appearance: none; width: 22px; height: 22px; border-ra
input[type=range]::-moz-range-thumb { ... } /* Firefox needs its own */
```

Related: `::before` / `::after` inject decorative content; pseudo-classes like `:hover`, `:last-child`, `:disabled` match by state/position.

In the code: the goal slider on StartAWouldBe. **PRACTICE 06**

13 · Borders, radius, shadows, pills

```
border-radius: 999px; /* fully-rounded pill (NO INCUMBENT, chips) */
box-shadow: 0 12px 34px rgba(16,16,20,.06); /* soft lift on cards */
box-shadow: 0 0 0 5px rgba(201,164,55,.22); /* a RING (no offset/blur) around the slider knob */
```

`box-shadow: x y blur spread color`. A shadow with only spread and no blur reads as a solid ring — used for the slider thumb and the rose caret.

14 · filter vs backdrop-filter

`filter: blur(4px)` blurs the **element itself** (used to blur the page behind the Qualify modal). `backdrop-filter: blur(10px)` blurs whatever is **behind** a translucent element (the frosted sticky nav). Easy to confuse; they do opposite things.

In the code: `.blurred (modal) = filter ; .wbNav (header) = backdrop-filter .` **PRACTICE 07**

15 · Media queries

```
@media (max-width: 900px) {  
  .grid { grid-template-columns: 1fr; } /* 3-up collapses to a stack */  
  .wbNav-search { display: none; }  
}
```

A breakpoint re-declares only what must change below a width. Because `fr` and `clamp()` already flex, you need fewer of these than you'd think.

16 · Map of the real files

Concept	Where it lives
flex split, min-width:0, card sizing, hover transition	<code>WouldBeRows.css</code>
compact timeline (rail/pins/YOU), gradient, absolute	<code>WouldBeRows.css</code> (<code>.dl*</code>)
modal blur (<code>filter</code>) + full-height column	<code>Wouldbe.css</code>
scoped tokens, grid fr, clamp, positioned timeline, scoped slider	<code>StartAWouldBe.css</code>
sticky nav, backdrop-filter, margin-left:auto	<code>WouldBeNavHeader.css</code>
self-contained panel, wrapping URL, moved-from-inline classes	<code>Regulations.css</code>

Practice set: `wouldbeandstartawouldbepactice` · pair each section with its `css/NN-*.html` · answers in `SOLUTIONS.md`